

基于随机计算的模拟器使用说明

概述

这个模拟器提供了一套基于**随机计算**的操作。它使用**概率比特流**和一组逻辑门（AND、OR、XOR、XNOR、NOT、MUX）来模拟传统比特的乘法和加法，适用于低精度的硬件加速器设计，来实现对深度学习算法的片上加速。

特性：

- 随机逻辑门**：通过使用标准逻辑门（AND、OR、XOR等）对随机比特流进行算术和逻辑操作。
- 查找表输出**：通过查找表来模拟每个逻辑门操作，根据指定的输入概率，直接跨过繁琐的马尔科夫链的迭代过程，模拟实际硬件的输出结果。
- 能量和功耗估算**：模拟器提供了基于硬件的能量、面积、延迟和功率的估算，并支持部分随机计算与传统方法的对比。
- 神经网络模块取代**：封装了基于不同逻辑门的加法器和乘法器，可被用于神经网络模块的随机计算加速构建。（示例：**随机池化** 和 **随机卷积**）

函数说明

1. `load_lookup_table(mode, p1, p2, scale_input, scale_mux, bit_length)`

加载指定逻辑门的查找表，并根据输入的概率 `p1`、`p2` 和比例因子模拟实际硬件上的输出结果。

参数：

- `mode`：选择要模拟的逻辑门类型，支持 'AND'、'OR'、'XOR'、'NOT'、'MUX'、'XNOR'。
- `p1`、`p2`：输入的概率值，范围为 $[0, 1]$ 。
- `scale_input`：输入概率的缩放因子，范围为 $[0.1, 1]$ 。
- `scale_mux`：MUX操作的缩放因子，范围为 $[0.0, 1.0]$ 。
- `bit_length`：用于表示随机数的位数，支持 8、16、32、64、128、256 等选择。

返回：

- 返回模拟后的输出概率，范围为 $[0, 1]$ 。

使用建议：

- 该函数会对输入的 `p1`、`p2` 和返回结果都引入一个基于比特流长度 $\text{bit_length}=2^n$ 的量化误差。 n 即代表了比特流的精度。同时 `bit_length` 的取值也会引入编码误差。
- `scale_input` 的取值越小，比特流的编码误差会越小；但过小的取值会引入量化误差。
- `bit_length` 越大，随机计算的准确率越高，但取值在 2^8 以上的时候在硬件上已经失去响应速度的优势了。

mode	概率表达式
AND	$P_{AND} = P_1 \times P_2$
OR	$P_{OR} = P_1 + P_2 - P_1 \times P_2$

mode	概率表达式
------	-------

NOT	$P_{NOT} = 1 - P_1$
-----	---------------------

XOR	$P_{XOR} = P_1 \times (1 - P_2) + (1 - P_1) \times P_2$
-----	---

MUX	$P_{MUX} = S \times P_1 + (1 - S) \times P_2$
-----	---

2. `sc_approx_adder(p1, p2, scale_input=1.0, bit_length=256, PPA_mode=False)`

使用随机计算OR门进行近似加法运算。

参数：

- `p1`、`p2`：输入的概率值，范围为 $[0, 1]$ 。
- `scale_input`：输入概率的缩放因子，范围为 $[0.1, 1]$ 。
- `bit_length`：随机计算使用的位数（默认是 256）。
- `PPA_mode`：表征硬件优势（默认是 `False`）。

返回：

- 返回加法结果的概率。

使用建议：

- 基于OR逻辑门的加法器会无法避免引入 $P_1 \times P_2$ 的计算误差，建议用于输入概率值较低的情况。
-

3. `sc_scaled_adder(p1, p2, scale_input=1.0, scale_factor=0.5, bit_length=256, PPA_mode=False)`

使用MUX实现进行加权计算。`scale_factor= 0.5` 时，输出为两输入的平均。

参数：

- `p1`、`p2`：输入的概率值，范围为 $[0, 1]$ 。
- `scale_input`：输入概率的缩放因子，范围为 $[0.1, 1]$ 。
- `scale_factor`：Mux的控制信号概率，默认为0.5。
- `bit_length`：随机计算使用的位数（默认是 256）。
- `PPA_mode`：表征硬件优势（默认是 `False`）。

返回：

- 返回加法结果的概率。

使用建议：

- 不同于基于OR逻辑门的加法，MUX在做为加法器时不会引入逻辑误差，但输出存在一个缩放。两输入之间的权重完全由`scale_factor`来决定，非常适合于平均池化。当MUX的控制信号`scale_factor`由`p1`和`p2`比较产生时，其可作为一个天然的最大池化函数。
-

4. `sc_multipler(p1, p2, scale_input=1.0, bit_length=256, PPA_mode=False)`

使用随机计算AND门进行乘法运算。

参数：

- `p1`、`p2`：输入的概率值，范围为 `[0, 1]`。
- `scale_input`：输入概率的缩放因子，范围为 `[0.1, 1]`。
- `bit_length`：随机计算使用的位数（默认是 256）。
- `PPA_mode`：表征硬件优势（默认是 `False`）。

返回：

- 返回乘法结果的概率。
-

5. `sc_avg_pooling(input_matrix, pool_size=2, stride=2, scale_input=1.0, bit_length=256)`

使用MUX门进行构建随机平均池化函数。

参数：

- `input_matrix`：一个2D NumPy数组，表示待池化的特征图。
- `pool_size`：池化窗口的大小（默认是 `2x2`）。
- `stride`：池化操作的步长（默认是 `2`）。
- `scale_input`：输入概率的缩放因子（默认是 `1.0`）。
- `bit_length`：随机计算使用的位数（默认是 256）。

返回：

- 返回一个2D NumPy数组，表示应用随机池化后的特征图。
-

6. `sc_convolution(input_matrix, kernel, stride=1, scale_input=1.0, scale_mux=0.5, bit_length=256)`

使用AND门进行乘法，MUX门进行加法的方式进行随机卷积。

参数：

- `input_matrix`：一个2D NumPy数组，表示输入特征图。
- `kernel`：一个2D NumPy数组，表示卷积核。
- `stride`：卷积操作的步长（默认是 `1`）。
- `scale_input`：输入概率的缩放因子（默认是 `1.0`）。
- `scale_mux`：用于MUX门的缩放因子（默认是 `0.5`）。
- `bit_length`：随机计算使用的位数（默认是 256）。

返回：

- 返回一个2D NumPy数组，表示应用随机卷积后的特征图。
-

使用示例

```

# 示例 1: 概率逻辑计算
import numpy as np
from sc import sc_avg_pooling
scale_input = 0.1
scale_mux = 0.5
bit_length = 128

for i in range(10):
    p1, p2 = np.random.rand(2)
    simulated_sum = load_lookup_table(
        mode='MUX',
        p1=p1,
        p2=p2,
        scale_input=scale_input,
        scale_mux=scale_mux,
        bit_length=bit_length
    )
    print(f"p1={np.round(p1, 6)}, p2={np.round(p2, 6)}: {simulated_sum}, real
value: [{np.round(scale_input * (p1 * scale_mux + p2 * (1 - scale_mux)), 6)}]")

```

Output:

```

p1=0.461696, p2=0.221462: [0.03125], real value: [0.034158]
p1=0.031403, p2=0.032632: [0.0078125], real value: [0.003202]
p1=0.869406, p2=0.156598: [0.0625], real value: [0.0513]
p1=0.908671, p2=0.931332: [0.0859375], real value: [0.092]
p1=0.707308, p2=0.927322: [0.046875], real value: [0.081732]
p1=0.805671, p2=0.521911: [0.0625], real value: [0.066379]
p1=0.966516, p2=0.400979: [0.03125], real value: [0.068375]
p1=0.059455, p2=0.206458: [0.03125], real value: [0.013296]
p1=0.423061, p2=0.146345: 0, real value: [0.02847]
p1=0.615699, p2=0.51586: [0.0390625], real value: [0.056578]

```

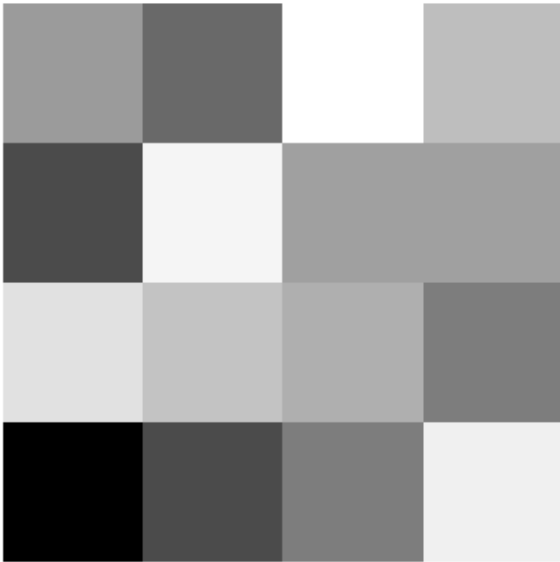
```

import numpy as np
from sc import sc_avg_pooling

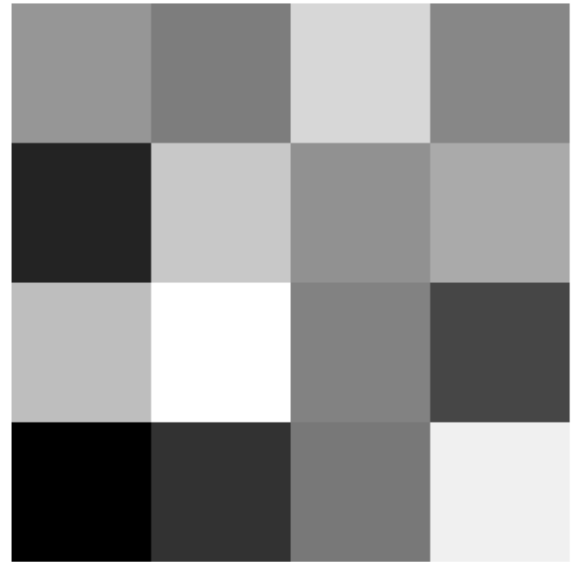
# 示例 2: 进行随机平均池化
input_matrix = np.random.rand(8, 8) # 随机生成 8x8 的输入矩阵
pooled_matrix = sc_avg_pooling(input_matrix, pool_size=2, stride=2,
scale_input=1.0, bit_length=256)

```

SC-Pooling

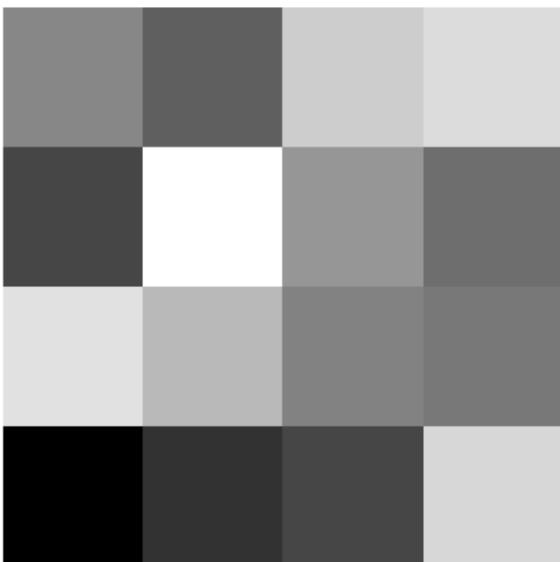


Conv-Pooling

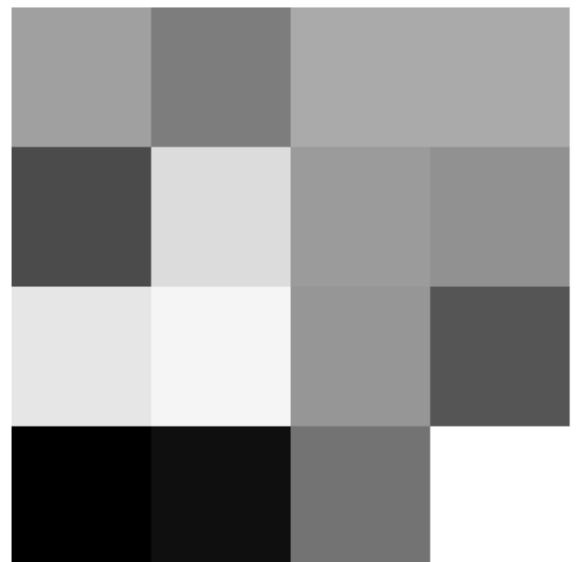


```
import numpy as np
from sc import sc_convolution
# 示例 3: 进行随机卷积 (量化误差+SC误差)
input_matrix = np.random.rand(8, 8) # 随机生成 8x8 的输入矩阵
kernel = np.random.rand(3, 3) # 随机生成 3x3 的卷积核
output_matrix = sc_convolution(input_matrix, kernel, stride=1, scale_input=1.0,
scale_mux=0.5, bit_length=256)
```

SC-Convolution



Conv-Convolution



伊辛模型

本文档将介绍P-bit的行为函数，以及基于P-bit的伊辛模型的相关函数，以最大割问题为实例提供了传统模拟退火，P-bit异步模拟退火，和同步模拟退火的代码演示。

1. Class IsingModel

初始化方法

__init__(self, size, Weight=None, Field=None)

初始化伊辛模型。通过给定的大小、耦合矩阵和外部磁场来创建一个伊辛模型。

- size** (int): 伊辛模型的大小，即P-bit的数量。
- Weight** (ndarray, 可选): P-bit之间的耦合矩阵。如果为None，则自动生成随机耦合矩阵。
- Field** (ndarray, 可选): 外部磁场。如果为None，则使用零场。

随机生成耦合矩阵

generate_random_Weight(self)

生成一个随机对称耦合矩阵。矩阵的值在-1到1之间，并且不包含自耦合（对角线为0）。

- 返回值: 对称耦合矩阵，矩阵的大小为size x size，且没有自耦合。

2. 计算方法

计算能量导数

calculate_Derivative(self, state=None)

计算伊辛模型状态的能量导数，表示由于与其他P-bit的相互作用和外部场的影响而对每个P-bit施加的力。

- state** (ndarray, 可选): 需要计算导数的特定状态。如果为None，则使用当前状态。
- 返回值: 能量导数向量，表示每个P-bit的力。

计算总能量

calculate_energy(self)

计算伊辛模型的总能量。能量由相互作用能量（基于耦合矩阵）和外部场能量组成。

- 返回值: 总能量（标量）。

3. Sigmoid函数

Sigmoid函数

sigmoid(self, x, beta=1, mu=0)

用于计算P-bit的切换概率的Sigmoid函数。将输入的x（通常是电压）映射到一个0到1之间的概率值。

- x (float): 输入值（电压或相关量），影响P-bit的状态。
- beta (float, 可选): 温度相关的参数，控制曲线的陡峭度，beta值越大，曲线越陡峭。
- mu (float, 可选): 偏移量，调整曲线的中心（默认为0）。
- 返回值: 概率值，表示P-bit切换的概率。

Sigmoid函数的反函数

inverse_sigmoid(self, y, beta, mu=0)

反Sigmoid函数，返回与给定概率y对应的输入值x。

- y (float): Sigmoid函数的输出概率（介于0和1之间）。
- beta (float): Sigmoid函数的陡峭度参数。
- mu (float, 可选): 偏移量，默认为0。
- 返回值: 对应概率y的输入值x。

4. 状态更新与能量景观

更新P-bit的状态

update_State(self, beta)

根据计算的能量导数和Sigmoid概率曲线更新P-bit的状态。每个P-bit根据其切换的概率进行更新。

- beta (float): 温度相关的参数，控制Sigmoid曲线的陡峭度，影响切换概率。

计算能量景观

energy_landscape(self)

计算伊辛模型的能量景观，涉及计算所有可能状态的能量。

- 返回值:
 - state_list: 所有可能的状态（以二进制表示）。
 - energy_list: 每个状态对应的能量。

5. 可视化方法

绘制耦合矩阵

plot_weight(self, real_index=True)

绘制伊辛模型的耦合矩阵。

- real_index (bool, 可选): 是否使用实际的索引标签。默认为True，表示使用实际索引。

绘制能量视图

`plot_energy_landscape(self)`

绘制伊辛模型的能量景观图，展示每个可能状态的能量。

6. 模拟退火法

异步模拟退火

`ising_simulated_annealing_Maxcut_Asyn(self, J, steps=1000, T_start=10, T_end=0.1, k=1)`

使用异步模拟退火法求解最大割问题，给定相互作用矩阵 J ，找到最小化能量的P-bit配置。通过电压（温度）实现模拟退火过程。异步定义为每次最多更新一个P-bit。

- J (dict): 相互作用矩阵，键为节点对 (i, j) ，值为相互作用项。
- `steps` (int, 可选): 模拟退火的步数（默认值1000）。
- `T_start` (float, 可选): 起始温度（默认值10）。
- `T_end` (float, 可选): 结束温度（默认值0.1）。
- `k` (float, 可选): 接受概率的缩放因子。
- 返回值:
 - `spins`: 每个节点的最终自旋状态。
 - `current_energy_list`: 每步模拟退火过程中的能量列表。
 - `temperature_list`: 每步模拟退火过程中的温度列表。

同步模拟退火

`ising_simulated_annealing_Maxcut_Syn(self, J, steps=1000, T_start=10, T_end=0.1, k=1)`

使用同步模拟退火法求解最大割问题，给定相互作用矩阵 J ，找到最小化能量的P-bit配置。同步指同一时刻更新所有的P-bits。

- J (dict): 相互作用矩阵，键为节点对 (i, j) ，值为相互作用项。
- `steps` (int, 可选): 模拟退火的步数（默认值1000）。
- `T_start` (float, 可选): 起始温度（默认值10）。
- `T_end` (float, 可选): 结束温度（默认值0.1）。
- `k` (float, 可选): 接受概率的缩放因子。
- 返回值:
 - `spins`: 每个节点的最终自旋状态。
 - `current_energy_list`: 每步模拟退火过程中的能量列表。
 - `temperature_list`: 每步模拟退火过程中的温度列表。


```
ising_simulated_annealing(J, steps=1000, T_start=10, T_end=0.1)
```

对给定的伊辛模型相互作用矩阵 J 执行模拟退火，目标是通过退火找到最小化能量的自旋配置。

参数：

- J (dict)：相互作用矩阵，每个键是节点对 (i, j) ，值是相互作用项。
- $steps$ (int, 可选)：模拟退火的步数（默认值是1000）。
- T_{start} (float, 可选)：起始温度（默认值是10）。
- T_{end} (float, 可选)：结束温度（默认值是0.1）。

返回值：

- $spins$ (dict)：最终的自旋配置。
- $current_energy_list$ (list)：每步模拟退火过程中的能量列表。
- $temperature_list$ (list)：每步模拟退火过程中的温度列表。

```
ising_simulated_annealing_with_penalty(J, h, nodes, steps=1000, T_start=10, T_end=0.1)
```

对带有平衡惩罚的伊辛模型进行模拟退火。惩罚项用于鼓励平衡分割。

参数：

- J (dict)：相互作用矩阵。
- h (dict)：每个节点的外部场。
- $nodes$ (list)：节点列表。
- $steps$ (int, 可选)：模拟退火步数（默认值是1000）。
- T_{start} (float, 可选)：起始温度（默认值是10）。
- T_{end} (float, 可选)：结束温度（默认值是0.1）。

返回值：

- $spins$ (dict)：最终的自旋配置。
- $current_energy$ (float)：最小化后的能量。

组合优化问题求解-最大割实列

1. 问题生成

```
generate_random_graph(n, p, seed=None, node_color='lightblue', font_weight='bold', edge_color='gray')
```

使用ER模型生成一个随机图并进行可视化。

2. 最大割问题求解

`brute_force_max_cut(G)`

使用暴力破解法求解最大割问题。寻找能够最大化两个分割之间边的数量的割。

`brute_force_max_cut_balanced(G)`

求解最大割问题，并且带有平衡分割的约束。即要求两个分割的大小相等。

3. 数据转换（哈密顿量映射）

`graph_to_adjacency_matrix(G)`

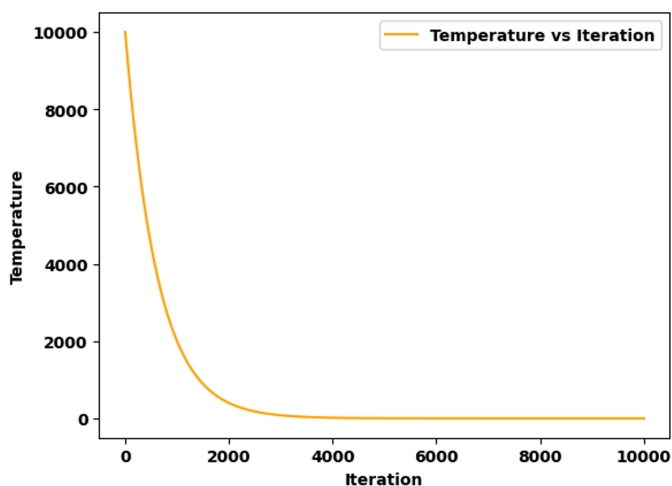
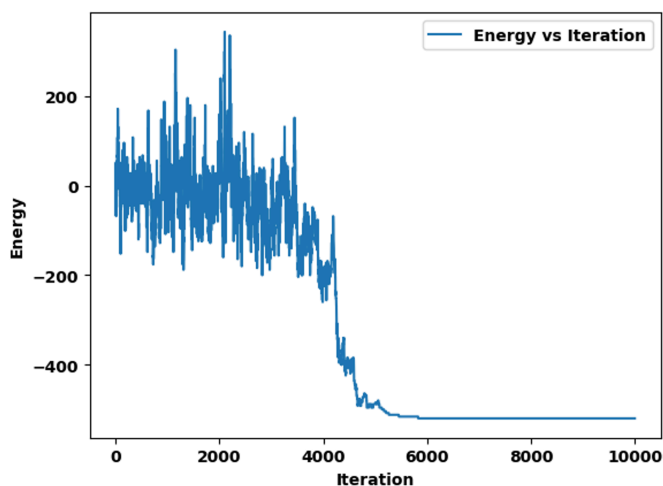
将networkx图转换为邻接矩阵。

`max_cut_to_ising(G)`

将最大割问题转换为伊辛模型的相互作用矩阵（J）。每条边在最大割问题中对应伊辛模型中的负相互作用。

使用示例

```
# 示例 1: 传统模拟退火求解
import numpy as np
from src_Ising import Ising_model, Max_cut
scale_input = 0.1
scale_mux = 0.5
bit_length = 128
G = Max_cut.generate_random_graph(80, 0.5)
J = Max_cut.max_cut_to_ising(G)
iteration_value = 10000
spins, current_energy_list, temperature_list =
Max_cut.ising_simulated_annealing(J, steps=iteration_value, T_start=10000.0,
T_end=0.001)
set1, set2 = Max_cut.convert_spins_to_cut(spins)
```



示例 2: P-bit 异步退火

```
import numpy as np
from src_Ising import Ising_model, Max_cut

P_bit_model = Ising_model.IsingModel(size=80)
J = Max_cut.max_cut_to_ising(G)
# print("Interaction Matrix J:\n", J)
iteration_value = 4000
spins, current_energy_list, temperature_list =
P_bit_model.ising_simulated_annealing_Maxcut_Asyn(J, steps=iteration_value,
T_start=100000, T_end=0.00001, k=10)
set1, set2 = Max_cut.convert_spins_to_cut(spins)
```

```
import numpy as np
from src_Ising import Ising_model, Max_cut
# 示例 3: P-bit 同步退火
P_bit_model = Ising_model.IsingModel(size=80)
J = Max_cut.max_cut_to_ising(G)
# print("Interaction Matrix J:\n", J)
iteration_value = 1000
spins, current_energy_list, temperature_list =
P_bit_model.ising_simulated_annealing_Maxcut_Syn(J, steps=iteration_value,
T_start=100000, T_end=0.0001, k=10)
set1, set2 = Max_cut.convert_spins_to_cut(spins)
```

